Kubernetes: YAML-манифесты

Занятие 04 · День 2, после обеда

Синтаксис YAML, структура манифеста, apply, Kustomize, проверка

Почему YAML

- Kubernetes API принимает **JSON**, но людям удобнее **YAML** (YAML — надмножество JSON)
- Манифесты — это **код**: хранятся в Git, проходят review, применяются из CI
- Тот же YAML дальше на курсе: **Ansible**, **Helm**, `.gitlab-ci.yml`

> Научиться уверенно читать и писать YAML — половина успеха в DevOps.

Синтаксис YAML: основы

```yaml
# комментарий
name: shop-api          # строка
replicas: 3             # число
enabled: true           # булево
nothing: null

ports:                  # список
  - 80
  - 443

labels:                 # словарь
  app: shop
  tier: backend

inline_list: [a, b, c]
inline_map: {cpu: 100m, memory: 64Mi}
```

Правила

- Отступы — **только пробелы** (обычно 2), табы запрещены
- Вложенность определяется **отступом**
- `-` начинает элемент списка
- `ключ: значение` — пробел после двоеточия обязателен
- Чувствительно к регистру

Многострочные строки и несколько документов

```yaml
script: |            # literal: переносы сохраняются
  echo "строка 1"
  echo "строка 2"

description: >       # folded: переносы → пробелы
  Длинный текст,
  который склеится в одну строку.
---                  # разделитель документов: несколько объектов в одном файле
apiVersion: v1
kind: Service
```

`|-` / `>-` — без завершающего перевода строки.

Ловушки YAML

Написали	Получили	Как правильно
`version: 1.10`	число `1.1`	`version: "1.10"`
`country: no`	`false` (YAML 1.1!)	`country: "no"`
`port: 0800`	восьмеричное / ошибка	`"0800"`
`cmd: echo: hi`	ошибка разбора	`cmd: "echo: hi"`
табуляция в отступе	ошибка	только пробелы
`env value: 8080`	число, а env ждёт строку	`value: "8080"`

Значения переменных окружения, аннотаций и меток — **всегда строки**. Сомневаетесь — ставьте кавычки.

Якоря и ссылки

```yaml
defaults: &resources          # & — объявить якорь
  requests: { cpu: 100m, memory: 64Mi }
  limits:   { memory: 128Mi }

api:
  resources: *resources       # * — подставить
worker:
  resources:
    <<: *resources            # слить и переопределить
    limits: { memory: 256Mi }
```

Kubernetes их понимает (раскрываются при разборе), но в манифестах встречаются редко. Зато часто — в `.gitlab-ci.yml` (день 5).

Структура любого манифеста

```yaml
apiVersion: apps/v1          # группа/версия API
kind: Deployment             # тип объекта
metadata:                    # кто это
  name: shop-api
  namespace: shop
  labels: { app.kubernetes.io/name: shop-api }
  annotations: {}
spec:                        # чего хотим (пишем мы)
  replicas: 2
  ...
status:                      # что есть (пишет кластер — в файлы не кладём)
  readyReplicas: 2
```

Где искать поля: `kubectl explain`

```
kubectl explain deployment
kubectl explain deployment.spec.strategy.rollingUpdate
kubectl explain pod.spec.containers.livenessProbe --recursive | head -30
kubectl explain httproute --api-version=gateway.networking.k8s.io/v1
```

- Работает для встроенных объектов и **CRD**
- Документация всегда соответствует **вашей версии кластера**

Не пишем с нуля: генерация

```
kubectl create deployment shop-api --image=nginx:1.28-alpine --replicas=2 \
  --dry-run=client -o yaml > deployment.yaml

kubectl create service clusterip shop-api --tcp=80:8080 \
  --dry-run=client -o yaml > service.yaml

kubectl create configmap shop-config --from-literal=MODE=prod \
  --dry-run=client -o yaml > configmap.yaml

kubectl create secret generic db --from-literal=password=x \
  --dry-run=client -o yaml          # base64 закодирует сам
```

Дальше — дописываем руками: ресурсы, пробы, метки.

create, apply, replace — в чём разница

Команда	Подход	Объект существует
`kubectl create -f`	императивный	ошибка `AlreadyExists`
`kubectl replace -f`	императивный	заменяет целиком
`kubectl apply -f`	**декларативный**	вычисляет разницу и применяет
`kubectl apply --server-side`	декларативный	слияние на сервере, учёт «владельцев» полей
`kubectl delete -f`	—	удаляет описанное в файле

```
kubectl apply -f k8s/                  # весь каталог
kubectl diff -f k8s/                   # что изменится — до применения
kubectl apply -f k8s/ --dry-run=server  # проверка на стороне API
```

Как apply понимает, что изменилось

- `kubectl apply` (client-side) хранит последний применённый вариант в аннотации `kubectl.kubernetes.io/last-applied-configuration`
- Трёхстороннее слияние: **файл** ↔ **last-applied** ↔ **текущий объект**
- **Server-side apply**: поле `managedFields` — кто каким полем владеет; конфликты, если два инструмента меняют одно поле

> Не смешивайте `kubectl edit` / `scale` и GitOps: ручные правки перезатрутся следующим `apply` — или вызовут конфликт.

Рекомендуемые метки

```yaml
metadata:
  labels:
    app.kubernetes.io/name: shop-api        # имя приложения
    app.kubernetes.io/instance: shop-api-prod
    app.kubernetes.io/version: "1.4.2"
    app.kubernetes.io/component: backend
    app.kubernetes.io/part-of: shop
    app.kubernetes.io/managed-by: helm      # kubectl | helm | kustomize
```

Их понимают Helm, Argo CD, Lens, дашборды. Selector держите **минимальным и неизменным** — у Deployment его нельзя поменять после создания.

Организация файлов

```
k8s/
├── namespace.yaml
├── configmap.yaml
├── secret.example.yaml      # пример! настоящий секрет — не в Git
├── deployment.yaml
├── service.yaml
└── kustomization.yaml
```

- Один объект — один файл (или логически связанная группа через `---`)
- Имена файлов — по `kind`
- Порядок применения: namespace → конфигурация → workloads → сервисы (`kubectl apply -f k8s/` идёт по алфавиту; повторный apply всё досоздаст)

Kustomize: окружения без копипасты

```
k8s/
├── base/                    # общее
│   ├── deployment.yaml
│   ├── service.yaml
│   └── kustomization.yaml
└── overlays/
    ├── staging/kustomization.yaml
    └── prod/kustomization.yaml
```

- **base** — общие манифесты, **overlays** — отличия окружений
- Встроен в kubectl: `kubectl apply -k`

Kustomize: overlay для production

```yaml
# overlays/prod/kustomization.yaml
resources: [../../base]
namespace: shop-prod
images:
  - name: shop-api
    newTag: "1.4.2"
patches:
  - target: { kind: Deployment, name: shop-api }
    patch: |-
      - op: replace
        path: /spec/replicas
        value: 3
```

```
kubectl kustomize k8s/overlays/prod     # посмотреть результат
kubectl apply -k k8s/overlays/prod      # применить
```

Kustomize vs Helm

	Kustomize	Helm
Подход	патчи поверх обычного YAML	шаблоны + значения
Встроен в kubectl	да (`-k`)	нет, отдельная утилита
Пакеты, репозитории	нет	да (чарты, OCI)
История релизов, rollback	нет	да
Порог входа	низкий	средний

На 5-й день упакуем приложение в **Helm-чарт** — так его будет деплоить GitLab CI.

Проверка манифестов до кластера

Инструмент	Что ловит
`yamllint`	синтаксис, отступы, дубли ключей
`kubectl apply --dry-run=server`	всё, что проверит API (нужен доступ к кластеру)
`kubeconform`	соответствие схемам Kubernetes (офлайн, быстро — для CI)
`kube-linter` , `Polaris` , `Checkov`	лучшие практики: нет limits, root, `latest`

```
yamllint k8s/
kubeconform -strict -summary k8s/
```

Все эти проверки отлично ложатся в стадию `test` пайплайна GitLab.

Best practices манифестов

- Явный тег образа, **никогда** `:latest`
- `resources.requests` (+ `limits.memory`) у каждого контейнера
- `readinessProbe` / `livenessProbe`
- `securityContext`: `runAsNonRoot: true`, `allowPrivilegeEscalation: false`
- Конфигурация — в ConfigMap, секреты — не в Git открытым текстом
- Рекомендуемые метки `app.kubernetes.io/*`
- `replicas ≥ 2` для stateless-сервисов + PodDisruptionBudget для важных
- Применять только через `apply` из Git (и CI), руками — только смотреть

ДЕМО Демо: манифест «с нуля» до работающего сервиса

```
mkdir -p ~/shop/k8s && cd ~/shop/k8s
kubectl create namespace shop --dry-run=client -o yaml > 00-namespace.yaml
kubectl -n shop create deployment shop-web --image=nginx:1.28-alpine --replicas=2 \
  --dry-run=client -o yaml > deployment.yaml
vim deployment.yaml          # метки, resources, probes, securityContext
kubectl -n shop create service loadbalancer shop-web --tcp=80:80 \
  --dry-run=client -o yaml > service.yaml

kubectl apply -f . --dry-run=server
kubectl diff -f .
kubectl apply -f .
kubectl -n shop get all
curl http://$(kubectl -n shop get svc shop-web -o jsonpath='{.status.loadBalancer.ingress[0].ip}')

sed -i 's/replicas: 2/replicas: 3/' deployment.yaml && kubectl diff -f . ; kubectl apply -f .
```

Эти файлы пригодятся завтра: положим их в Git.

Итоги

- YAML: пробелы, списки `-`, словари `ключ: значение`, **кавычки для строк-«чисел»**
- Манифест: `apiVersion`, `kind`, `metadata`, `spec` (`status` пишет кластер)
- `kubectl explain` + `--dry-run=client -o yaml` — быстрый старт
- `apply` + `diff` + `--dry-run=server` — безопасные изменения
- Kustomize — окружения без копипасты; Helm — пакеты и релизы

Практика 04

Пишем манифесты сами

Задание — на платформе testlms.kz